MOVIE RECOMMENDATION SYSTEM

A Complete Final Year Project Guide

(B.Tech — Artificial Intelligence & Machine Learning)

Scratch-to-final roadmap covering problem definition, dataset, algorithms, system design, implementation, evaluation, deployment, report structure and viva preparation.

Prepared as a reference guide — fill in your name, college, guide name and enrollment details on your final report's title page.

Table of Contents

- 1. Project Overview & Objectives
- 2. Problem Statement
- 3. Literature Survey (Key Points)
- 4. Dataset
- 5. System Requirements
- 6. Recommendation Approaches & Algorithms
- 7. Proposed System Architecture
- 8. Technology Stack
- 9. Module-wise Breakdown
- 10. Step-by-Step Implementation Plan
- 11. Core Code Walkthrough (Skeleton)
- 12. Evaluation Metrics
- 13. Deployment
- 14. 8–10 Week Project Timeline
- 15. Final Report / Documentation Structure
- 16. Viva & Presentation Questions
- 17. Future Scope & Enhancements
- 18. References & Resources

1. Project Overview & Objectives

A Movie Recommendation System is a filtering application that predicts and suggests movies a user is likely to enjoy, based on their past behaviour (ratings, watch history) and/or the behaviour of similar users. It is one of the most popular final-year AIML projects because it combines data preprocessing, classical machine learning, optional deep learning, and a deployable web application — covering the full ML lifecycle in one project.

Objectives

- To study and implement multiple recommendation techniques: content-based filtering, collaborative filtering, and a hybrid approach.
- To preprocess and analyze a real-world movie ratings dataset (MovieLens).
- To build similarity/prediction models that recommend top-N movies to a user.
- To evaluate model performance using standard metrics (RMSE, MAE, Precision@K, Recall@K).
- To deploy the trained model as an interactive web application using Flask/Streamlit.
- To document the system in a formal project report suitable for university evaluation.

Scope

The project can be scaled to the time and skill level available: a minimum-viable version uses content-based or item-based collaborative filtering on the MovieLens-100K dataset with a simple UI. An advanced version adds matrix factorization (SVD), a hybrid model, a Neural Collaborative Filtering model in PyTorch/TensorFlow, and a cloud-deployed web app with posters fetched from the TMDB API.

2. Problem Statement

With thousands of movies available on streaming platforms, users face information overload and find it difficult to discover content matching their taste. The goal of this project is to design and implement a system that analyzes historical user ratings and movie metadata to predict user preferences and generate a personalized, ranked list of movies a given user has not yet watched.

Formal definition: Given a set of users U, a set of movies M, and a sparse user-item rating matrix R (where R[u][m] is the rating user u gave movie m, mostly unknown), predict the missing entries of R and recommend the top-N movies with the highest predicted rating for a target user.

3. Literature Survey (Key Points to Cite in Report)

Your report should include a short literature survey (5–8 papers/sources) comparing prior approaches. Summarize each with: technique used, dataset, and reported result/limitation. Suggested areas to read:

- **Collaborative Filtering** — classic user-based and item-based neighborhood methods (Resnick et al., Sarwar et al. on item-based CF for Amazon-scale recommendations).
- **Matrix Factorization** — Koren, Bell & Volinsky, "Matrix Factorization Techniques for Recommender Systems" (2009) — the key paper behind the Netflix Prize-winning SVD approach.
- **Content-Based Filtering** — TF-IDF / cosine similarity over movie genres, cast, and plot keywords.
- **Hybrid Recommenders** — combining content and collaborative signals to solve the cold-start problem.
- **Deep Learning based** — He et al., "Neural Collaborative Filtering" (2017); Autoencoder-based CF.

- **Cold-Start Problem** — strategies for new users/new items with no rating history.

4. Dataset

Use the **MovieLens** dataset from GroupLens Research — the de facto standard benchmark for recommender systems, free for academic/educational use.

Dataset	Size	Best for
MovieLens 100K (ml-100k)	100,000 ratings, 943 users, 1,682 movies	Quick prototyping, laptop-friendly
MovieLens 1M (ml-1m)	1 million ratings, 6,000 users, 4,000 movies	More realistic evaluation
MovieLens 25M (ml-25m)	25 million ratings, 162,000 users, 62,000 movies	Advanced / deep learning models
TMDB 5000 Movie Dataset	Metadata: overview, cast, crew, genres, keywords	Content-based filtering

Recommended: Start with MovieLens 100K or 1M for collaborative filtering, and merge with the TMDB 5000 dataset (join on movie title/IMDB id) for content-based filtering and to fetch posters for your UI. Download links:

- https://grouplens.org/datasets/movielens/
- https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

Key Files (ml-100k / ml-1m)

File	Contents
u.data / ratings.csv	user_id, movie_id, rating (1–5), timestamp
u.item / movies.csv	movie_id, title, release date, genres
u.user / users.csv	user_id, age, gender, occupation, zip code

5. System Requirements

Hardware

- A standard laptop/PC — 8 GB RAM minimum (16 GB recommended if training Neural CF on ml-25m).
- GPU is optional; only needed for the deep-learning extension (Google Colab's free GPU is sufficient).

Software

- Python 3.9+
- Jupyter Notebook / Google Colab for experimentation
- VS Code or PyCharm for building the final application
- Git & GitHub for version control

6. Recommendation Approaches & Algorithms

6.1 Content-Based Filtering

Recommends movies similar to ones a user already liked, based on movie attributes (genre, cast, director, plot keywords/overview). Steps: combine metadata into a single 'tags' text field per movie, vectorize using TF-IDF or CountVectorizer, then compute pairwise cosine similarity between all movies. For a given movie, recommend the N most similar movies.

Pros: No cold-start for new users with even one liked movie; explainable. **Cons:** Limited diversity, cannot capture taste it hasn't seen in metadata.

6.2 Collaborative Filtering (CF)

Uses the user-item ratings matrix only — no metadata needed. Two classic sub-types:

- **User-based CF:** find users with similar rating patterns (cosine/Pearson similarity), recommend movies that similar users rated highly.
- **Item-based CF:** compute similarity between items based on how users rated them; recommend items similar to ones the target user already rated highly. Generally more stable and scalable than user-based.

6.3 Matrix Factorization (Model-Based CF)

Decomposes the sparse rating matrix R (users × movies) into two lower-dimensional latent factor matrices, R ≈ P x Q^T, where P is user-factors and Q is item-factors. Learned using SVD, or gradient-descent / ALS optimization (as in the Surprise library's SVD implementation). This is the single most effective classical technique and is strongly recommended as your core model.

6.4 Hybrid Recommender

Combines content-based and collaborative scores (e.g., weighted sum, or use content-based only for new users/items and switch to CF once enough ratings exist). Solves the cold-start problem and typically improves accuracy — a good 'advanced' feature to highlight in your report/viva.

6.5 Deep Learning Extension (Optional, for higher grading)

Neural Collaborative Filtering (NCF): replace the dot-product in matrix factorization with a multi-layer perceptron that learns a non-linear user-item interaction function, using embedding layers for user_id and movie_id. Implementable in PyTorch or TensorFlow/Keras with 3–4 dense layers.

7. Proposed System Architecture

A typical end-to-end pipeline for your report's architecture diagram:

1. **Data Layer:** Raw MovieLens + TMDB CSV files.
2. **Preprocessing Layer:** cleaning, merging datasets, handling missing values, feature engineering (tags column for content-based; encoding user_id/movie_id for CF).
3. **Model Layer:** Content-based similarity matrix + Collaborative Filtering model (SVD) + optional Hybrid combiner / Neural CF model.
4. **Recommendation Engine:** takes a user_id or movie title as input, returns ranked Top-N movie list.
5. **Application Layer (Backend):** Flask/FastAPI REST API exposing a /recommend endpoint.
6. **Presentation Layer (Frontend):** Streamlit or HTML/CSS/JS UI showing movie posters (via TMDB API) and titles for the recommended list.
7. **Deployment Layer:** Hosted on Streamlit Community Cloud / Render / Heroku with the trained model pickled and loaded at runtime.

Draw this as a labeled block diagram (boxes + arrows) in your report using draw.io, PowerPoint, or Canva — reviewers expect a visual system architecture diagram, not just text.

8. Technology Stack

Layer	Recommended Tools
Language	Python 3.x
Data Handling	Pandas, NumPy
ML / Similarity	scikit-learn (TF-IDF, cosine_similarity), Surprise (SVD, KNNBasic)
Deep Learning (optional)	PyTorch or TensorFlow/Keras (for Neural CF)
Visualization / EDA	Matplotlib, Seaborn
Backend API	Flask or FastAPI
Frontend / UI	Streamlit (fastest) or React/HTML-CSS-JS
External Data	TMDB API (for movie posters and metadata)
Model persistence	pickle / joblib
Version control	Git + GitHub
Deployment	Streamlit Community Cloud, Render, or Heroku

9. Module-wise Breakdown

1. **Data Collection Module** — download and load MovieLens + TMDB datasets.
2. **Data Preprocessing Module** — clean nulls/duplicates, merge datasets on movie title/id, engineer the combined 'tags' field, encode categorical IDs.
3. **Exploratory Data Analysis (EDA) Module** — rating distribution, most-rated movies/genres, sparsity of the ratings matrix, user activity distribution.
4. **Content-Based Model Module** — TF-IDF vectorization + cosine similarity matrix + recommend_by_title() function.
5. **Collaborative Filtering Module** — train/test split of ratings, SVD model training via Surprise, predict() and recommend_for_user() functions.
6. **Hybrid Module (optional)** — weighted combination of content and CF scores.
7. **Evaluation Module** — compute RMSE, MAE, Precision@K, Recall@K on the held-out test set.
8. **API / Backend Module** — Flask/FastAPI endpoints wrapping the trained model.
9. **Frontend / UI Module** — search box + recommendation grid with posters.
10. **Deployment Module** — containerize/host the app, environment config, requirements.txt.

10. Step-by-Step Implementation Plan

1. **Set up environment** — create a virtual environment, install pandas, numpy, scikit-learn, scikit-surprise, flask/streamlit.
2. **Download datasets** — MovieLens (ratings.csv, movies.csv) and TMDB 5000 (movies + credits CSV).
3. **Load & explore data** — check shapes, nulls, data types; plot rating distribution and genre counts.
4. **Clean & merge** — merge TMDB movies + credits on movie id/title; extract genres, cast (top 3), director, and keywords from the JSON-like columns using ast.literal_eval.
5. **Build 'tags' column** — concatenate overview + genres + keywords + cast + director into one text field per movie; lowercase and remove extra spaces.
6. **Content-based model** — vectorize tags with TfidfVectorizer(stop_words='english', max_features=5000); compute cosine_similarity(matrix, matrix); write a function that returns the top-N most similar movies to a given title.
7. **Collaborative filtering model** — load ratings.csv into a Surprise Dataset/Reader; split train/test (80/20); train SVD(); evaluate with accuracy.rmse() and accuracy.mae().
8. **Top-N generation for CF** — for a target user, predict ratings for all unrated movies, sort descending, return top-N.
9. **(Optional) Hybrid** — normalize both score types to 0–1 and combine, e.g. final_score = 0.6*CF_score + 0.4*content_score.
10. **(Optional) Neural CF** — build an embedding-based MLP model in PyTorch/Keras; train on the same ratings data; compare RMSE against SVD.
11. **Evaluate all models** — tabulate RMSE/MAE/Precision@K/Recall@K for each approach in your report.
12. **Save the final model** — pickle the similarity matrix / trained SVD model + a small movies dataframe.
13. **Build the API/UI** — Streamlit app: search/select a movie or log in as a user_id, click 'Recommend', display poster grid using TMDB API for images.

14 **Test end-to-end** — verify recommendations look sensible for a few sample users/movies.

15 **Deploy** — push to GitHub, deploy on Streamlit Community Cloud (free) or Render.

16 **Write the report & prepare slides** — using the structure in Section 15.

11. Core Code Walkthrough (Skeleton)

Below is a minimal working skeleton to get you started. Expand each part per the steps above.

11.1 Content-Based Filtering

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies = pd.read_csv("tmdb_movies_clean.csv")  # has a 'tags' column

tfidf = TfidfVectorizer(stop_words='english', max_features=5000)
tfidf_matrix = tfidf.fit_transform(movies['tags'])
similarity = cosine_similarity(tfidf_matrix)

def recommend_content(title, n=10):
    idx = movies[movies['title'] == title].index[0]
    scores = list(enumerate(similarity[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)[1:n+1]
    return movies.iloc[[i for i, _ in scores]]['title'].tolist()
```

11.2 Collaborative Filtering (SVD via Surprise)

```
from surprise import Dataset, Reader, SVD, accuracy
from surprise.model_selection import train_test_split

ratings = pd.read_csv("ratings.csv")  # userId, movieId, rating
reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)

trainset, testset = train_test_split(data, test_size=0.2, random_state=42)
model = SVD(n_factors=50, n_epochs=20, lr_all=0.005, reg_all=0.02)
model.fit(trainset)

predictions = model.test(testset)
print("RMSE:", accuracy.rmse(predictions))
print("MAE:", accuracy.mae(predictions))

def recommend_cf(user_id, n=10):
    all_movie_ids = ratings['movieId'].unique()
    seen = ratings[ratings.userId == user_id]['movieId'].tolist()
    unseen = [m for m in all_movie_ids if m not in seen]
    preds = [(m, model.predict(user_id, m).est) for m in unseen]
    preds.sort(key=lambda x: x[1], reverse=True)
    return preds[:n]
```

11.3 Simple Hybrid Combiner

```
def hybrid_score(cf_score, content_score, alpha=0.6):
    # both scores should be normalized to [0, 1] beforehand
    return alpha * cf_score + (1 - alpha) * content_score
```

11.4 Streamlit App Skeleton

```
import streamlit as st
import pickle

movies = pickle.load(open("movies.pkl", "rb"))
similarity = pickle.load(open("similarity.pkl", "rb"))

st.title("Movie Recommendation System")
```

```
selected = st.selectbox("Pick a movie you like:", movies['title'].values)

if st.button("Recommend"):
    results = recommend_content(selected)
    cols = st.columns(5)
    for col, title in zip(cols, results[:5]):
        col.write(title)  # fetch poster via TMDB API and use col.image(url)
```

12. Evaluation Metrics

Metric	Measures	Used for
RMSE (Root Mean Squared Error)	Average magnitude of rating prediction error	Rating prediction (CF/SVD)
MAE (Mean Absolute Error)	Average absolute prediction error	Rating prediction (CF/SVD)
Precision@K	Fraction of top-K recommended items that are relevant	Top-N ranking quality
Recall@K	Fraction of relevant items captured in top-K	Top-N ranking quality
F1-Score@K	Harmonic mean of Precision@K and Recall@K	Overall ranking quality
Coverage	% of catalog the system is able to recommend	Diversity check

Present these as a comparison table across your models (Content-Based vs Item-CF vs SVD vs Hybrid vs Neural CF) in the Results chapter — this comparison is usually the centerpiece of the evaluation section and a common viva question.

13. Deployment

1. Wrap the trained model and the recommend function into a Streamlit app (fastest for a student project) or a Flask REST API + simple HTML/JS frontend.
2. Create a requirements.txt listing exact package versions used.
3. Push the code (excluding large raw datasets — use .gitignore) to a public GitHub repository.
4. Deploy for free on Streamlit Community Cloud (share.streamlit.io) by linking your GitHub repo, or on Render/Railway for a Flask app.
5. Optionally register a free TMDB API key to fetch and display real movie posters in the UI.
6. Add the live demo link and GitHub repo link on your report's cover/abstract page.

14. Suggested 8–10 Week Project Timeline

Week	Task
1	Topic finalization, literature survey, synopsis submission
2	Environment setup, dataset download, initial EDA
3	Data cleaning & preprocessing, feature engineering ('tags' column)
4	Content-based filtering model + testing
5	Collaborative filtering (SVD) model + testing
6	Evaluation (RMSE/MAE/Precision@K/Recall@K), model comparison
7	Hybrid model / optional Neural CF; build Streamlit/Flask UI
8	Integration, end-to-end testing, deployment
9	Report writing, diagrams, screenshots
10	PPT preparation, mock viva, final submission

15. Final Report / Documentation Structure

Most Indian university AIML final-year report formats follow this chapter structure — check your own department's format guidelines and adjust headings accordingly:

1. Title Page (project title, your name, roll no., guide name, college, university, year)
2. Certificate (signed by guide/HOD)
3. Acknowledgement
4. Abstract (150–250 words summarizing problem, method, and result)
5. Table of Contents, List of Figures, List of Tables
6. **Chapter 1 — Introduction:** background, motivation, problem statement, objectives, scope
7. **Chapter 2 — Literature Survey:** summary of existing systems/papers with a comparison table
8. **Chapter 3 — System Analysis:** existing system limitations, proposed system, feasibility study (technical/economic/operational), requirement specification (SRS)
9. **Chapter 4 — System Design:** architecture diagram, DFD (Level 0/1), ER diagram / dataset schema, UML use-case and sequence diagrams
10. **Chapter 5 — Methodology / Algorithms:** detailed explanation of content-based, collaborative, hybrid approach with math/formulas used
11. **Chapter 6 — Implementation:** module descriptions, key code snippets, tools used
12. **Chapter 7 — Testing:** test cases table, unit/integration testing approach
13. **Chapter 8 — Results & Discussion:** screenshots of the working UI, evaluation metric tables/graphs, comparison of models
14. **Chapter 9 — Conclusion & Future Scope**
15. References (IEEE format recommended)
16. Appendix (full code listing, if required by your department)

16. Viva & Presentation Questions (Practice These)

- What is the difference between content-based and collaborative filtering?
- What is the cold-start problem, and how does your system address it?
- Explain matrix factorization / SVD in the context of recommender systems.
- Why did you choose RMSE/MAE over accuracy for evaluating this system?
- What is the difference between Precision@K and Recall@K, and why does K matter?
- How does your item-based CF differ from user-based CF? Why is item-based often preferred at scale?
- How would you scale this system to millions of users (e.g., approximate nearest neighbors, ALS, distributed computing with Spark)?
- What is sparsity in the rating matrix, and how does it affect your model?
- How does Neural Collaborative Filtering differ from classical matrix factorization?
- What are the ethical concerns in recommendation systems (filter bubbles, popularity bias)?

17. Future Scope & Enhancements

- Incorporate implicit feedback (clicks, watch time, search history) instead of only explicit ratings.
- Add context-aware recommendations (time of day, device, mood/genre trends).
- Use sentiment analysis on user reviews as an additional signal.
- Add a Graph Neural Network (GNN)-based recommender for richer user-item relationship modeling.
- Real-time recommendation updates using streaming data (Kafka + online learning).
- A/B testing framework to compare recommendation strategies with live users.
- Multi-language / multi-region movie catalog support.

18. References & Resources

Datasets

- GroupLens MovieLens Datasets — https://grouplens.org/datasets/movielens/
- TMDB 5000 Movie Dataset (Kaggle) — https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

Key Papers

- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8).
- Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-Based Collaborative Filtering Recommendation Algorithms. WWW '01.
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T-S. (2017). Neural Collaborative Filtering. WWW '17.

Libraries & Tools

- scikit-learn — https://scikit-learn.org/
- Surprise (recommender library) — https://surpriselib.com/
- Streamlit — https://streamlit.io/
- TMDB API — https://developer.themoviedb.org/

Tip: build the minimum-viable version first (content-based + item-based CF + a simple Streamlit UI), get it fully working end-to-end, then layer on SVD, hybrid logic, and deep learning only if time permits. A small project that fully works beats a large one that's half-broken at submission time.